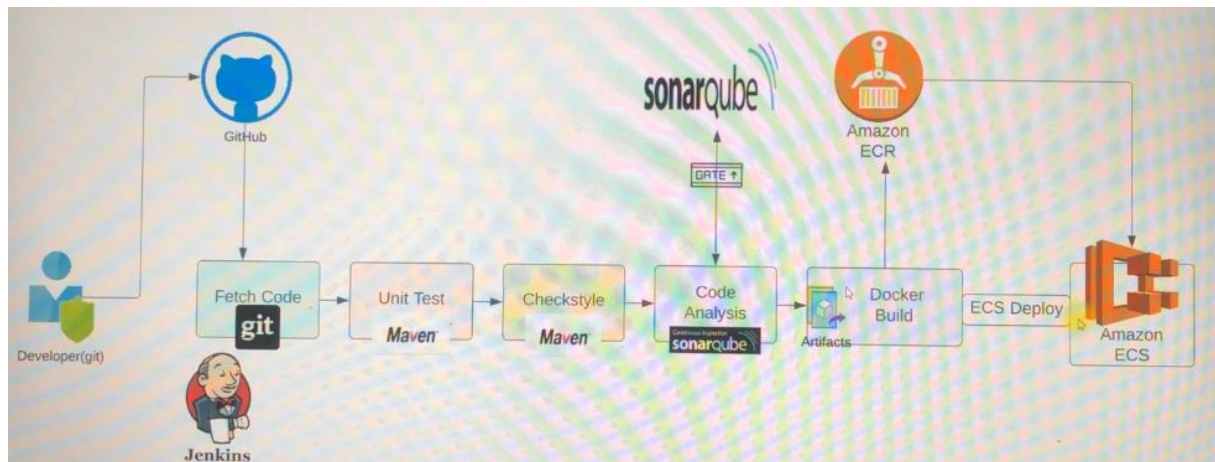


This documentation is done to describe all the process for building this pipeline using Jenkins



First step : install Jenkins

Firstly, launch an EC2 instance to run Jenkins

Name : jenkins-server

OS : Ubuntu

Amazon Machine Image : Ubuntu 24.04

Instance type : t2.small (we need at least 2Gio of memory)

Key pair : jenkins-key.pem

Security group : name(jenkins -sg), description(jenkins -sg)

Allow ssh inbound rule from MyIP

Allow Custom TCP with port 8080 from MyIP (To use it in the browser)

Launch the instance

Conect to the instance with ssh

Secondly, to install Jenkins, go the doc of Jenkins and get the command to install

Jenkins on a ubuntu server : click

here(<https://www.jenkins.io/doc/book/installing/linux/>)

This is step of installing jenkins on ubuntu

sudo apt update

sudo apt install openjdk-21-jdk -y

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
```

```
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc]" \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null
```

```
sudo apt-get update
```

```
sudo apt-get install jenkins -y
```

```
ubuntu@ip-172-31-81-198: ~
elidj@Sougoumay MINGW64 ~/OneDrive/Bureau/DevOps/aws/project/vprofile-project (jenkins_CI)
$ ssh -i "jenkins-key.pem" ubuntu@ec2-44-202-61-231.compute-1.amazonaws.com
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1029-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sun Jun 29 11:37:05 UTC 2025

System load:  0.65           Processes:            106
Usage of /:   25.8% of 6.71GB Users logged in:          0
Memory usage: 11%           IPv4 address for enx0: 172.31.81.198
Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

Last login: Sun Jun 29 11:36:21 2025 from 176.177.55.131
To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-81-198:~$ |
```

Switch as root user : `sudo -i`

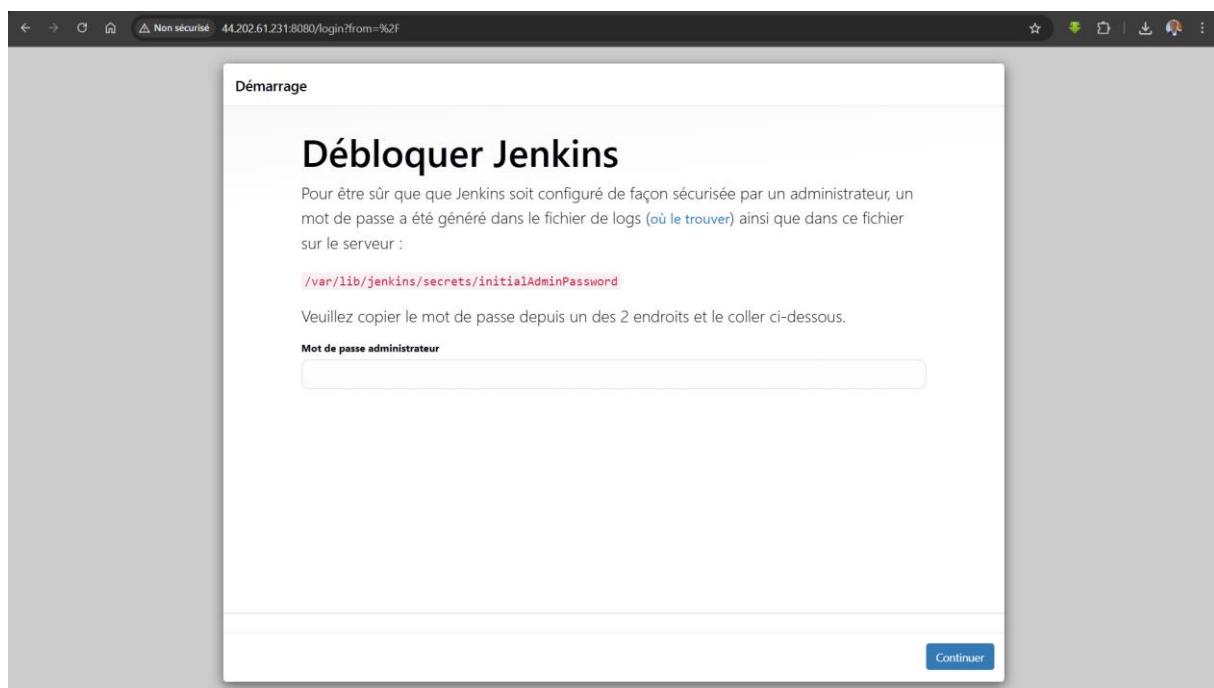
Execute the commands below to install Jenkins on the server

Check Jenkins status to be sure it's running

```
ubuntu@ip-172-31-81-198: ~
ubuntu@ip-172-31-81-198:~$ systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Sun 2025-06-29 11:43:46 UTC; 3min 32s ago
     Main PID: 4115 (java)
        Tasks: 37 (limit: 2332)
       Memory: 365.3M (peak: 384.2M)
          CPU: 19.041s
      CGroup: /system.slice/jenkins.service
              └─4115 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=>

Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: ec4a6d1c7a37470b9e76edf515a6901e
Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: This may also be found at: /var/lib/jenkins/secrets/in>
Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: *****
Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: *****
Jun 29 11:43:38 ip-172-31-81-198 jenkins[4115]: *****
Jun 29 11:43:46 ip-172-31-81-198 jenkins[4115]: 2025-06-29 11:43:46.320+0000 [id=31] INFO
Jun 29 11:43:46 ip-172-31-81-198 jenkins[4115]: 2025-06-29 11:43:46.348+0000 [id=24] INFO
Jun 29 11:43:46 ip-172-31-81-198 systemd[1]: Started jenkins.service - Jenkins Continuous Integration
Jun 29 11:43:46 ip-172-31-81-198 jenkins[4115]: 2025-06-29 11:43:46.786+0000 [id=48] INFO
Jun 29 11:43:46 ip-172-31-81-198 jenkins[4115]: 2025-06-29 11:43:46.790+0000 [id=48] INFO
ubuntu@ip-172-31-81-198:~$ ls /var/lib/jenkins/
config.xml      jobs            secret.key      updates
hudson.model.UpdateCenter.xml  nodeMonitors.xml secret.key.not-so-secret userContent
jenkins.telemetry.Correlator.xml plugins         secrets         users
ubuntu@ip-172-31-81-198:~$
```

Check to connect to the server on the browser with port 8080. The next image is the first step



To have the password of the administrator, execute the next command

```
cat /var/lib/jenkins/secrets/initialAdminPassword :
```

```
ec4a6d1c7a37470b9e76edf515a6901e
```

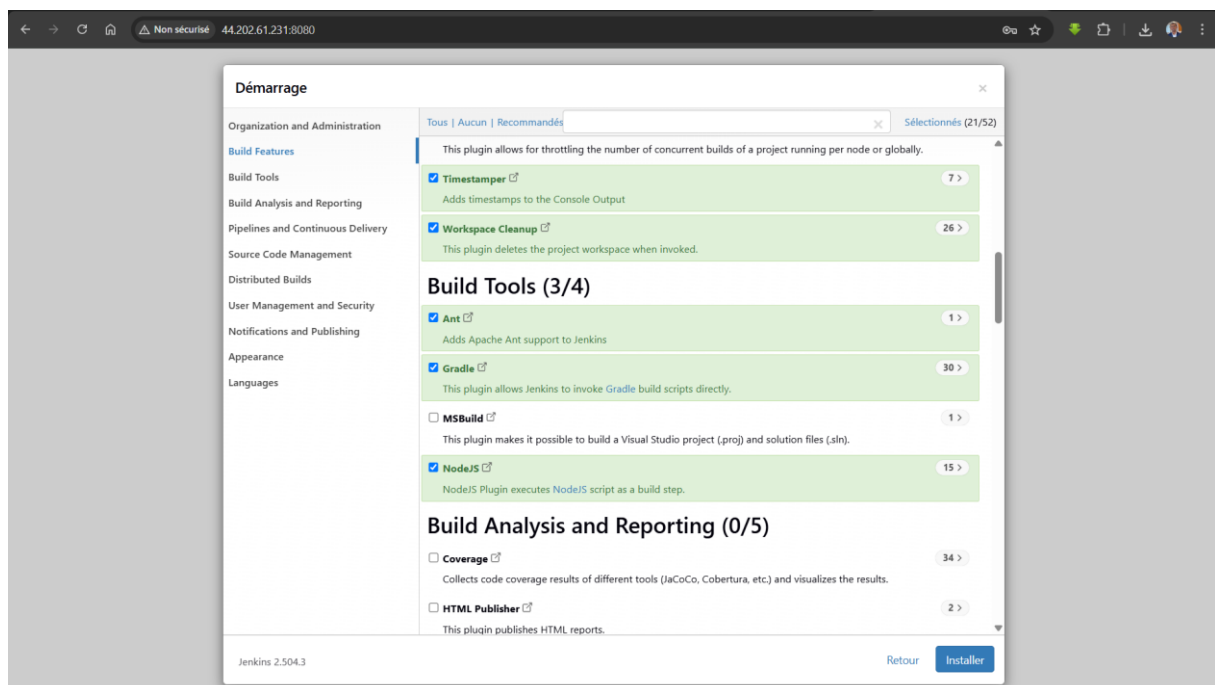
```
root@ip-172-31-81-198: ~
ubuntu@ip-172-31-81-198:~$ cat /var/lib/jenkins/secrets/initialAdminPassword
cat: /var/lib/jenkins/secrets/initialAdminPassword: Permission denied
ubuntu@ip-172-31-81-198:~$ cat /var/lib/jenkins/secrets/initialAdminPassword
cat: /var/lib/jenkins/secrets/initialAdminPassword: Permission denied
ubuntu@ip-172-31-81-198:~$ sudo -i
root@ip-172-31-81-198:~# cat /var/lib/jenkins/secrets/initialAdminPassword
ec4a6d1c7a37470b9e76edf515a6901e
root@ip-172-31-81-198:~#
```

The next step is this



Click to select plugin install

Select nodejs and click install



Installation en cours...

Installation en cours...

✓ Folders	✓ OWASP Markup Formatter	✓ Build Timeout	⌚ Credentials Binding	** Ionicons API
⌚ Timestampers	⌚ Workspace Cleanup	⌚ Ant	⌚ Gradle	Folders
⌚ Pipeline	⌚ GitHub Branch Source	⌚ Pipeline: GitHub Groovy Libraries	⌚ Pipeline Graph View	OWASP Markup Formatter
⌚ Git	⌚ SSH Build Agents	⌚ Matrix Authorization Strategy	☐ PAM Authentication	** ASM API
⌚ LDAP	⌚ Email Extension	⌚ Mailer	☐ Dark Theme	** JSON Path API
⌚ NodeJS				** Structs
				** Pipeline: Step API
				** Token Macro
				Build Timeout
				**ouncycastle API

Create the first admin user

Démarrage

Créer le 1er utilisateur Administrateur

Nom d'utilisateur

admin

Mot de passe

.....

Confirmation du mot de passe

.....

Nom complet

Admin

Adresse courriel

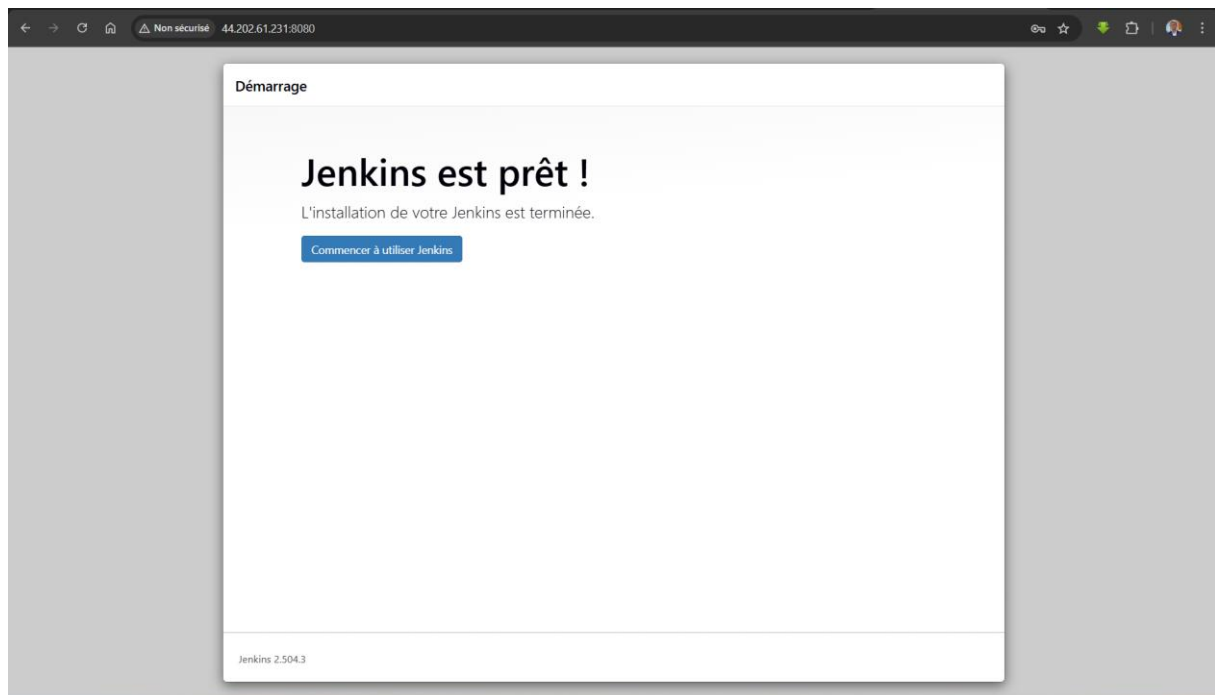
admin@net.com

Jenkins 2.504.3

Continuer en tant qu'Administrateur

Sauver et continuer

Choose default configuration of url and continue to get the next page



Finally, this is the dashboard

Installation of tools

In this step, we need to install some important tools for our exercise such as Maven and JDK. Follow the next steps :

Click to manage Jenkins

Click to tools

Scroll down and click to add JDK

Give the name JDK17

Uncheck install automatically if selected by default

To have Java_Home, connect to the server with ssh command

Switch to admin user : `sudo -i`

Update and install JDK17 : `apt update && apt install openjdk-17-jdk -y`

Execute the command to have jdk home : `ls /usr/lib/jvm/`

You will see two lines : `java-17-openjdk-amd64` and `java-21-openjdk-amd64`

The full path of JDK17 is : `/usr/lib/jvm/java-17-openjdk-amd64`

Give a name and choose the version 3.9.x of Maven in the select list

Click to save to finish

Second step : launch the sonar server

Firstly, we need to launch an EC2 instances with the next parameters :

Name : sonar-server

OS : Ubuntu

Amazon Machine Image : Ubuntu 24.04

Instance type : t2.medium (we need more resources than the micro type)

Key pair : sonar-key.pem

Security group : name(sonar-sg), description(sonar-sg)

Allow ssh inbound rule from MyIP

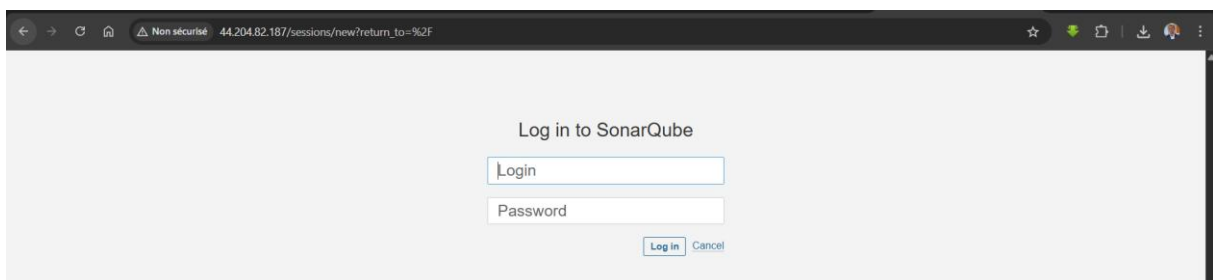
Allow Custom TCP with port 80 from MyIP (because we have nginx in the front)

Allow Custom TCP with port 80 from jenkins-sg with description Allow Jenkins to communicate

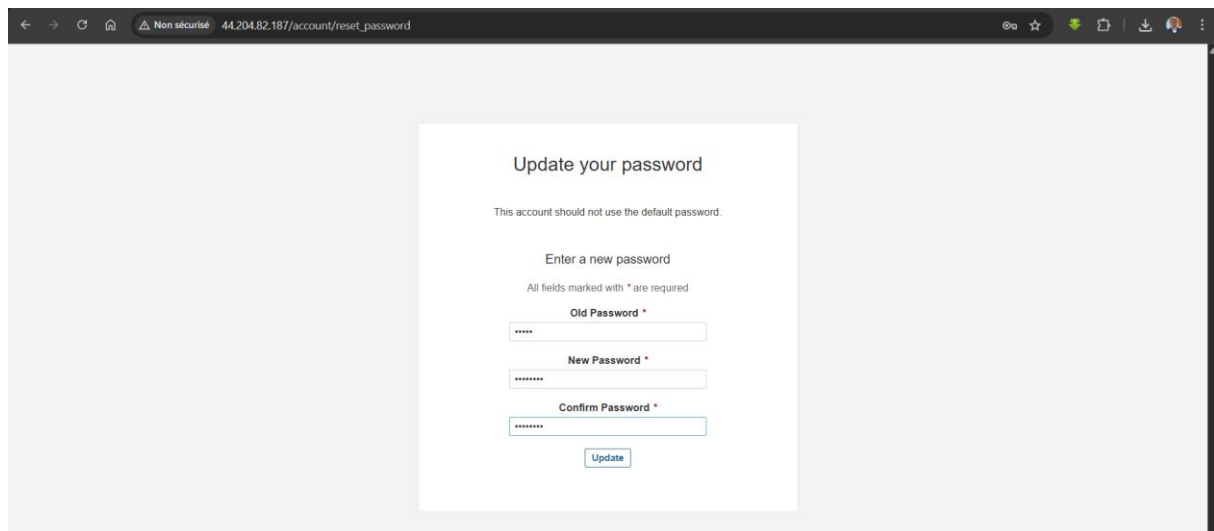
In the github repo, branch Jenkins_CI, copy the content of the file userdata/sonar-setup.sh and paste it toe User data file of Advance details section to setup-up the server while launching the instance

Launch the instance to get the server

Now check if sonar is running by using the browser (take the public IP address and paste in the browser, no need to specify port because the nginx front is running in port 80)

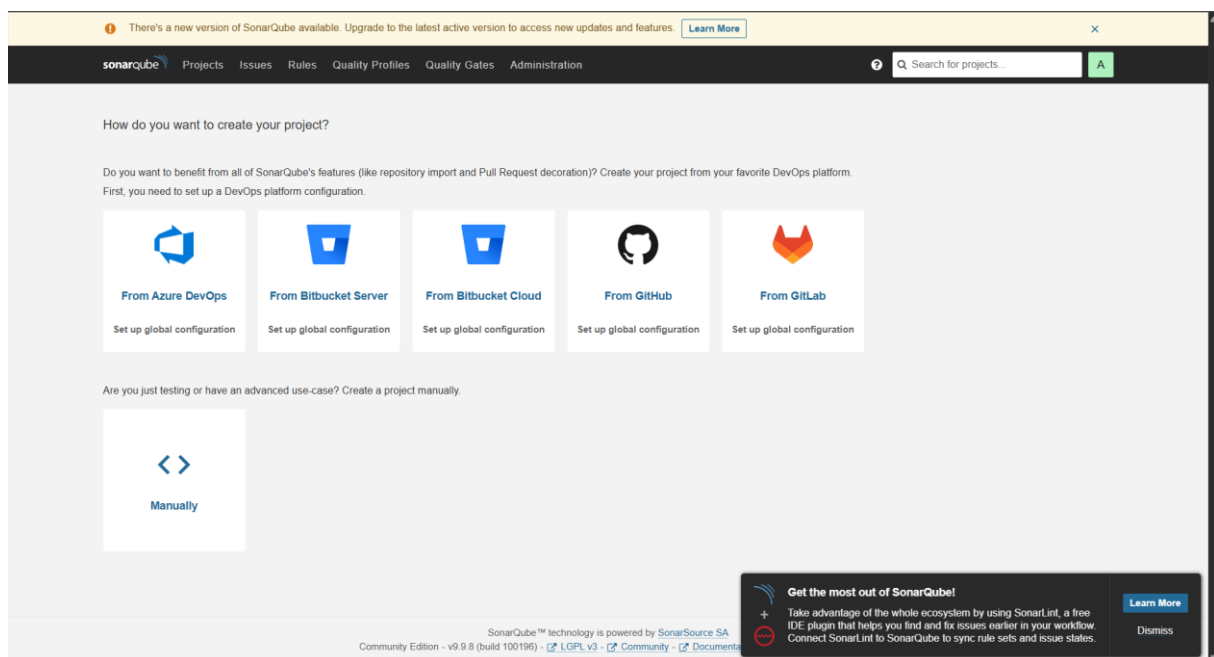


The login is admin and the password is admin



The next step is to change the password, I chose admin123

Now we are connected, this is the dashboard



Install Plugins for CI

In this step, we gonna install plugins for Sonar, BuildTimestamp, Pipeline Maven Integration.

To do that, connect into the Jenkins Server, go to Manage Jenkins, click into Manage Plugins, switch to available, search sonarqube scanner, nexus artifact uploader, build timestamp, pipeline maven integration, pipeline utility steps and check them all. Then click to install

The next is to execute a pipeline with the next code


```

pipeline {
    agent any

    tools {
        maven "MAVEN3.9"
        jdk "JDK17"
    }

    stages {
        stage('Fetch code') {
            steps {
                git branch: 'jenkins_CI',
                url: 'https://github.com/Sougoumay/vprofile-project'
            }
        }

        stage('Unit Test') {
            steps {
                sh 'mvn test'
            }
        }

        stage('Build') {
            steps {
                sh 'mvn install -DskipTests'
            }
            post {
                success {
                    echo 'Archiving artifact'
                    archiveArtifacts artifacts: '**/*.war'
                }
            }
        }
    }
}

```

Go to the dashboard (Jenkins in web)

New Item

Give a name

Select pipeline

Click OK

Paste the code on the script input

Execute

Save

Launch the build

Code Analysis

SonarQube Scanner tool

Sonar server details

From sonar we need token

To analyze the code with SonarQube, Jenkins and SonarQube must be in run. The we need three things :

SonarQube Scanner tool in Jenkins

Sonar server details in Jenkins

Token from Sonarqube to authorize Jenkins taking actions

First of all, to install Sonarqube Scanner, go to Jenkins dashboard

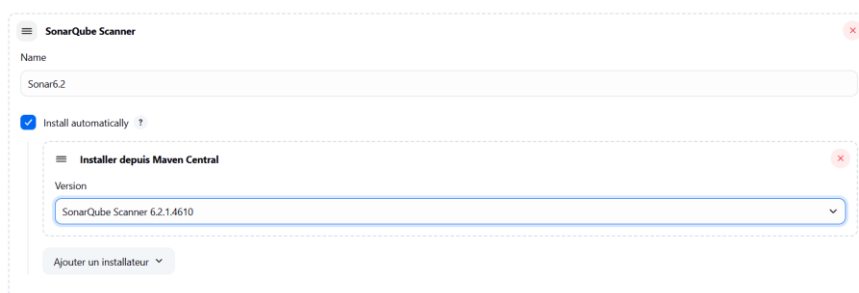
Click to manage Jenkins

Click to tools

Scroll down until SonarQube scanner and click to Add SonarQube Scanner

Give the name Sonar6.2

Choose the version 6.2.1.4610



The screenshot shows the 'SonarQube Scanner' configuration page in Jenkins. The 'Name' field is set to 'Sonar6.2'. The 'Install automatically' checkbox is checked. Below this, there is a section titled 'Installer depuis Maven Central' which contains a 'Version' dropdown menu. The dropdown menu is open, showing the selected version 'SonarQube Scanner 6.2.1.4610'. At the bottom of the configuration box, there is a button labeled 'Ajouter un installateur'.

In the next step, we need to store Sonarqube Server details in the Jenkins. SonarQube Server tool will scan the code and then we need to upload result to sonarqube server. For that we need to store the information of sonarqube server. Follow the next steps

Go to manage Jenkins

Go to System

Scroll down until you find Sonarqube server

Check environment variables

Click on Add Sonarqube

Give the name sonarserver

Server url will be the private ip of sonar, go to AWS, copy and paste (<http://private-ip>)

For the authentication we need Sonar token

Now connect to sonar server with public ip in the browser

Login with your credentials (admin and admin123)

In the top right click to the symbol A

Click to my account

Click to security

Give the name Jenkins

Select User Token

Then Generate

There's a new version of SonarQube available. Upgrade to the latest active version to access new updates and features. [Learn More](#)

sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration ? Search for projects... A

A Administrator Profile Security Notifications Projects

Tokens

If you want to enforce security by not providing credentials of a real SonarQube user to run your code scan or to invoke web services, you can provide a User Token as a replacement of the user login. This will increase the security of your installation by not letting your analysis user's password going through your network.

Generate Tokens

Name: Type: Expires in:

Name	Type	Project	Last use	Created	Expiration
No tokens					

New token "jenkins" has been created. Make sure you copy it now, you won't be able to see it again!

Copy

squ_bb14910c998b6209042353b8d62acd490ab4c55e

Name	Type	Project	Last use	Created	Expiration	
jenkins	User		Never	July 4, 2025	August 3, 2025	Revoke

Copy your token (if you olost, you have to remove this one and generate a new one)

Go to Jenkins

Click to add (in the authentication section of sonarqube servers(Jenkins))

Select the type secret text

Paste the token in the secret field

Give sonartoken as ID and description

Click to add

Then choose sonarqube in the list

Installations de SonarQube
Liste des installations de SonarQube

Nom

sonarserver

URL du serveur

Par défaut à http://localhost:9000

http://172.31.95.76

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonartoken

+ Ajouter

Avancé

Ajouter une installation SonarQube

Be sure to authorize Jenkins to communicate with sonar by adding an inbound rule on sonar security group, with type http

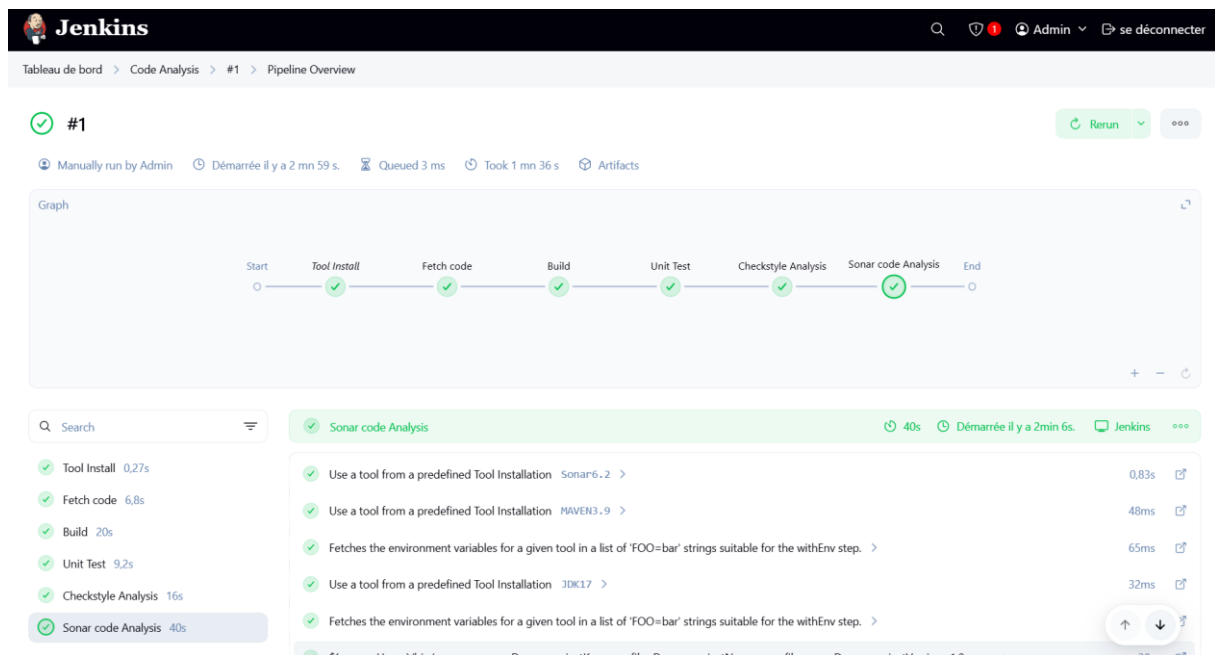
In the code of the pipeline we executed before, put the build stage before the unit test stage, then add this new stage

```
stage('Checkstyle Analysis') {
    steps {
        sh 'mvn checkstyle:checkstyle'
    }
}

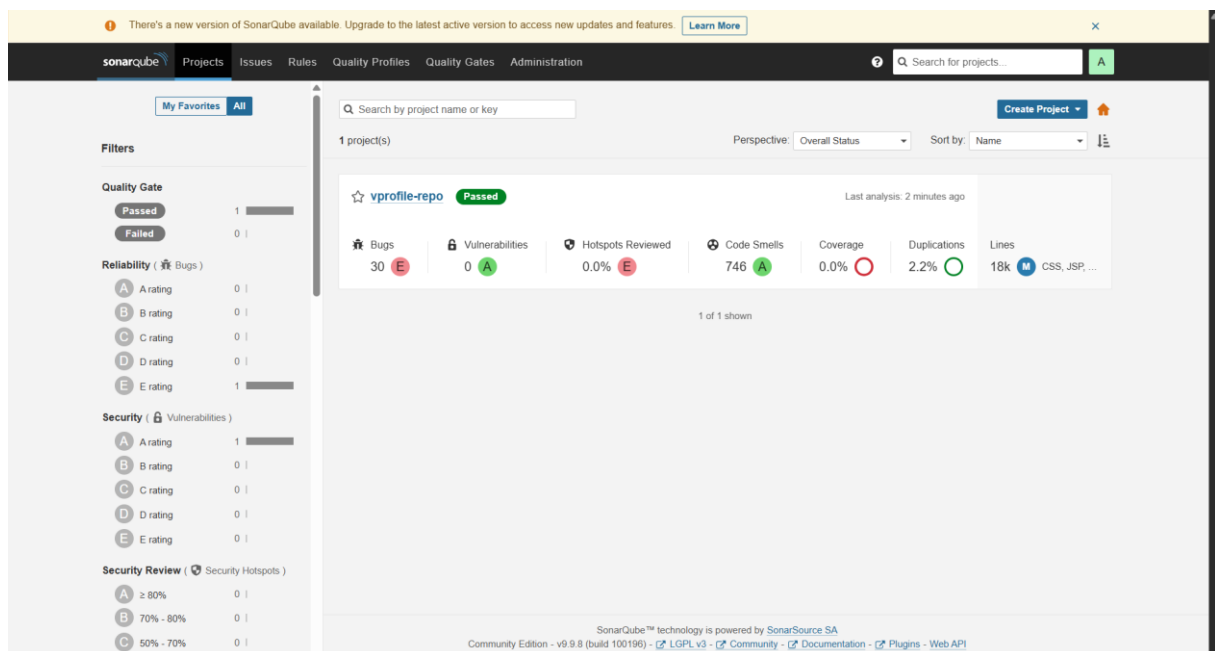
stage('Sonar code Analysis') {
    environment {
        scannerHome = tool 'Sonar6.2'
    }
    steps {
        withSonarQubeEnv('sonarserver') {
            sh "${scannerHome}/bin/sonar-scanner -Dsonar.projectKey=vprofile \
            -Dsonar.projectName=vprofile-repo \
            -Dsonar.projectVersion=1.0 \
            -Dsonar.sources=src/ \
            -Dsonar.java.binaries=target/test-classes/com/visualpathit/account/controllerTest/
            \
            -Dsonar.junit.reportsPath=target/surefire-reports/ \
            -Dsonar.jacoco.reportsPath=target/jacoco.exec \
            -Dsonar.java.checkstyle.reportPaths=target/checkstyle-result.xml"
        }
    }
}
```

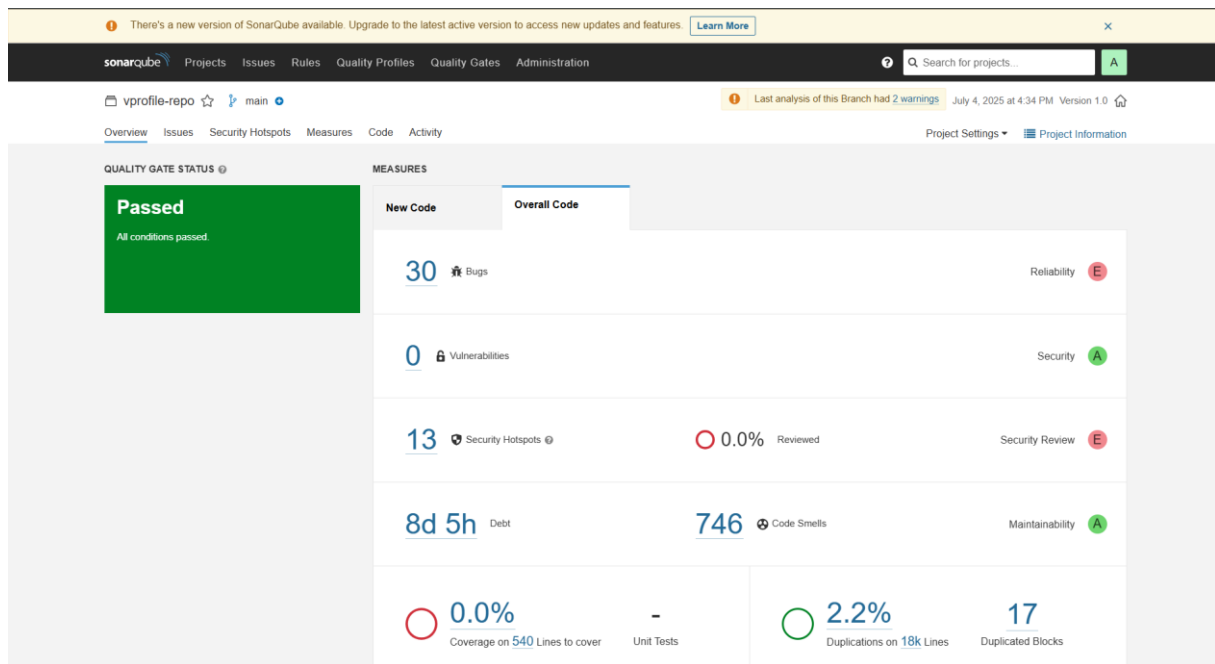
Now, create a new item (pipeline), copy paste this code and run the pipeline

Here we go, our pipeline is executed successfully



Go to sonar server and you will have this picture





Everything is good and the pipeline passed, this is the default quality gate. In the next step, we will define Quality Gate to determine when the analysis will be a success or not.

To define a quality gate,

Go to SonarQube dashboard

Go to quality gate section

Click to create button

Give a name : Jenkins-QG

Save to create the quality gate

Scroll Down

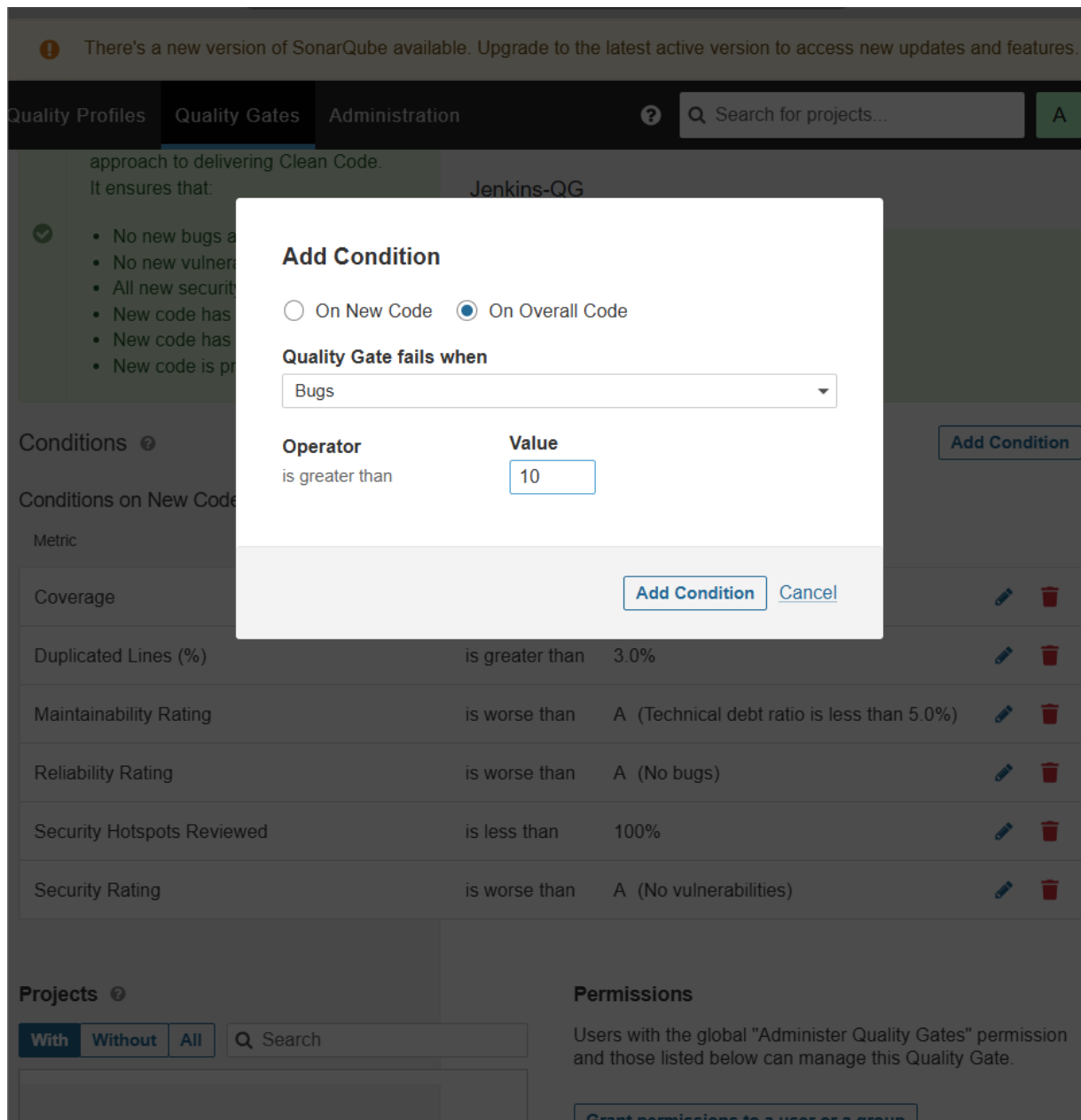
Click to Unlock editing

Click to Add Condition

Choose On Overall Code

In Quality Gate fails when, you have lots off condition, in this exercise, I choosed Bugs

Give 10 as Value (There are 30 bugs, so this time, the result will not pass)



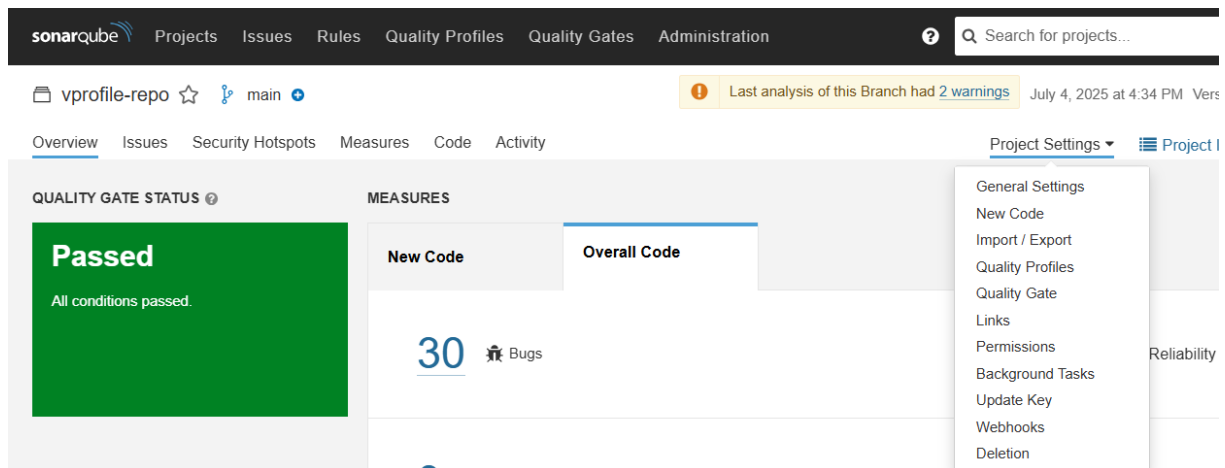
After that

Go to Projects Section

Choose our project (vprofile-repo)

Click to project settings

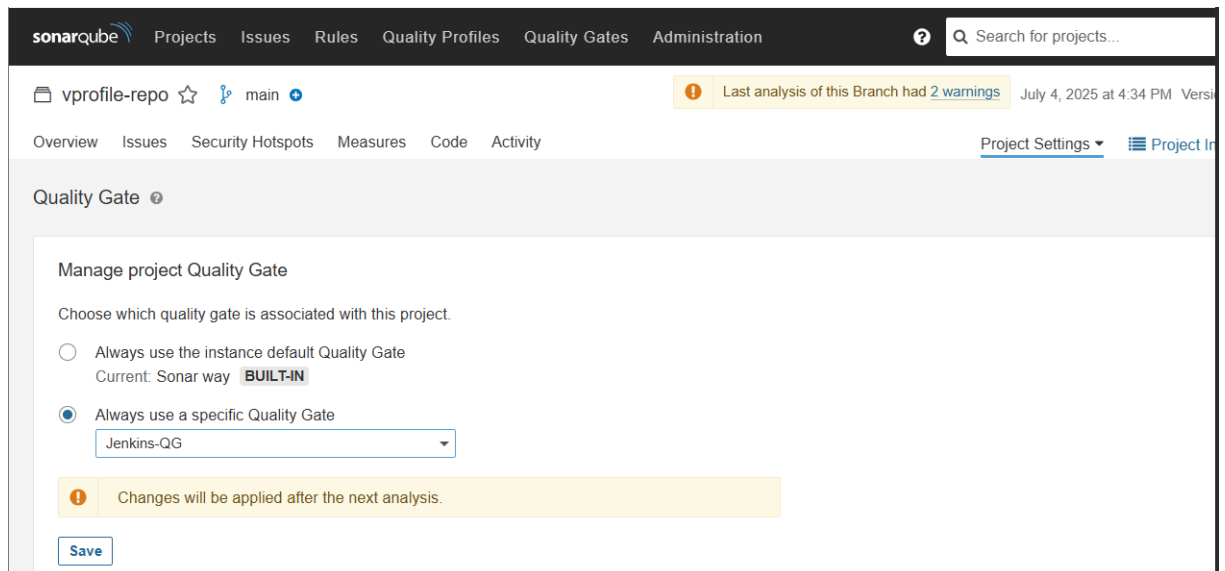
Click to quality gate



Choose Always use a specific quality gate

Select Jenkins-QG

Then Save



Once the quality gate is checked, sonarqube will sent the result to Jenkins. So we need to create a webhook. Still on Sonarqube,

choose the project,

click to project settings

lick to webhook to create one

Click to create

Give the name : Jenkins-CI-WebHook

The url will be <http://private-ip-of-jenkins:8080/sonarqube-webhook>

Click on create

Create Webhook

All fields marked with * are required

Name *



URL *



Server endpoint that will receive the webhook payload, for example: "http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example: "https://myLogin:myPassword@my_server/foo"

Secret

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

Create

[Cancel](#)

Add this stage on the pipeline

```
stage("Quality Gate") {  
    steps {  
        timeout(time: 1, unit: 'HOURS') {  
            waitForQualityGate abortPipeline: true  
        }  
    }  
}
```

```
}  
  
}
```

Update our pipeline code on Jenkins to adding this stage below, to do that

Go to the dashboard of Jenkins

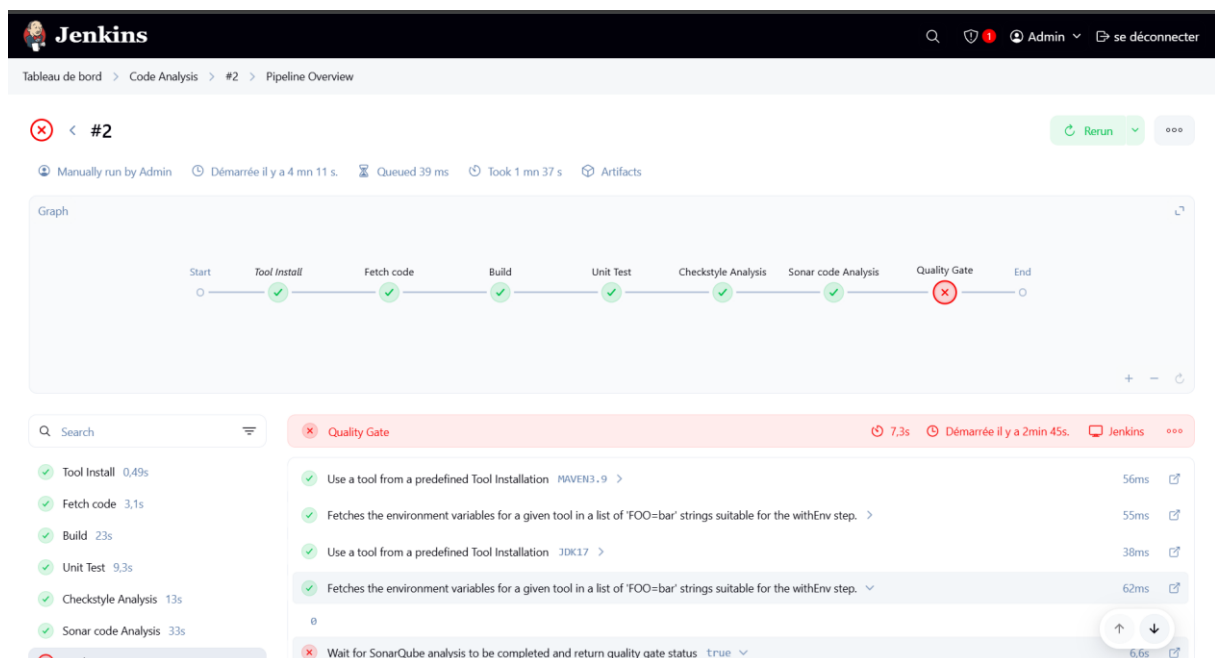
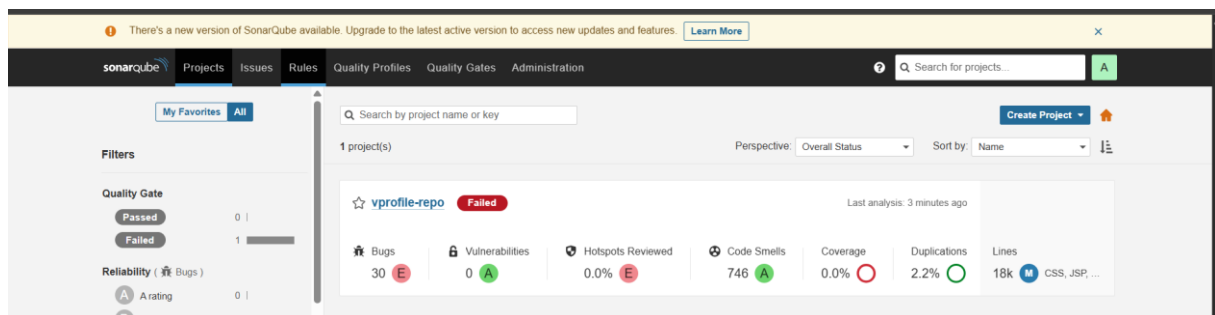
Click to the Analysis Code Pipeline

Click to configure

Scroll down, put the code at the end of the Sonar code analysis stage

Save and Run the build

Now, the pipeline will fail because there are more than 10 Bugs, SonarQube Analysis will not pass, so the last stage of the pipeline will fail



In the next step, we will store the artefacts in Nexus Reposiotry. To have a success pipeline, I switch the quality gate to the default.

Upload docker images to ECR and host to ECS

To build an image within Jenkins and push it to ecr, we need the next informations

- AWS
 - IAM USER with ACCESS Keys
 - ERC Registry (Docker registry from AWS)
- Plugins
 - Docker
 - Docker pipeline
 - Ecr
- Jenkins
 - Store AWS Access Keys(credentials)
 - Install Docker Engine in Jenkins

To it, we will follow the next steps

- Install AWS CLI
- Install docker engine in Jenkins
 - Add Jenkins user to docker group & reboot
- Create IAM User
- Create ECR repo
- Plugins
 - ECR, docker pipeline, AWS SDK for credentials
- Store AWS credentials in Jenkins
- Run the pipeline

Install AWS CLI on Jenkins

- Do a ssh to Jenkins
- Sudo apt update
- Sudo snap install aws-cli --classic
- Sudo -i
- Follow docker installation on ubuntu :
<https://docs.docker.com/engine/install/ubuntu/>
- Add Jenkins user into the docker group : `usermod -a -G docker Jenkins`
- Check Jenkins is added : `id Jenkins`

Create IAM user with Access Keys

- Go to aws management console
- Go to IAM services
- Click on create user
- Give the name Jenkins then next
- In the permissions policies choose Attach policies directly
 - Search container and check AmazonEC2ContainerRegistryFullAccess
 - Search ECS and check AmazonECS_FullAccess
- Click on next and create user
- Click on the user Jenkins
- Click to the security credentials section
- Scroll down and click on create access key
- Select CLI
- Check the confirmation
- Click on next and on create access key
- Download your credentials and keep them safe
- Click on done

Create the ECR Registry

- Search ECR and go the Amazon elastic container registry service
- Click on create
- Give the name vprofileappimg
- Click on create

Install All Plugins

Go to Jenkins on the browser

Click on manage Jenkins

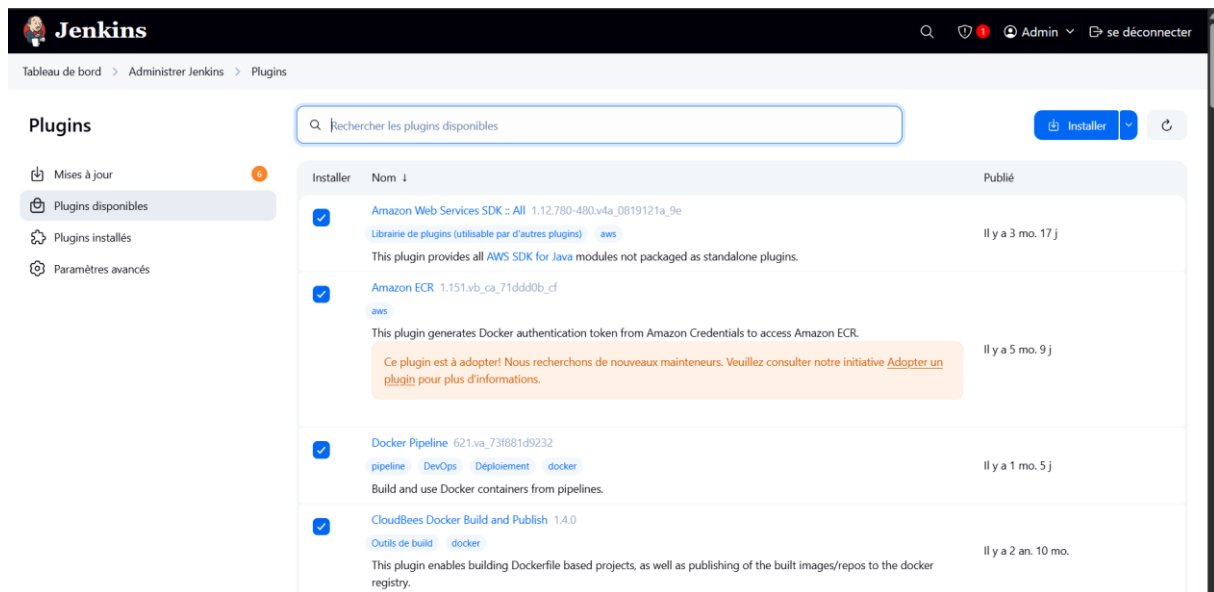
Click on Plugins

Search AWS and check AWS SDK :: All

Search ECR and check Amazon ECR

Search for docker plugin and check CloudBees Docker build and publish and Docker pipeline

Click on install



Add AWS Access Keys (credentials) to Jenkins and run the pipeline

- Go to Jenkins
- Click on manage Jenkins
- Click on credentials
- On the stores scoped to Jenkins, click on System
- Click on Global credentials
- Click on Add Credentials
- Click AWS Credentials on the type list
- Give to ID and description the name : awscreds
- Give your Access key ID and the secret access key from your credentials downloaded in the previous step
- Click on create

Now you can run the pipeline. The code is on my [github repo](#) in the branch docker

I had some problems, there is no space left to build the image. So I follow the next steps

- I created a snapshot from the volume of Jenkins
- I extended the size of the Jenkins' volume
- I followed all the step suggested by AWS
- I run again the pipeline
- Everything is ok now, so I deleted the snapshot

Deliver the docker images to ECS

To host docker images into ECS, we need to do the next tasks

- AWS
 - Create ECS cluster
 - Create ECS service
- Plugin
 - Plugin pipeline : aws steps

AWS ECS Setup

Create Cluster

Create a task definition

Create a service

If you go into the cluster vprofile we created, you can see a section named task and one other named service. In a cluster, a task mean a container. The role of a service is to manage the container, run automatically or take other actions in the container in the case of need. In this section, we will create a service to manage the container.

The next steps are to follow:

Go on the services section

Click on create

Select vprofileapptask on the task defition family

Service name is vprofileappsvc

In the deployment failure section, uncheck the use the amazon ECS deployment circuit breaker

In the network section, click on create new security group, this will be the security that w create four our container and the load balancer that we are creating

Give the name = description = vprofileapp-ecs-elb-sg

Add a rule for the load balancer (type = HTTP, source = anywhere). Load balancer is contacted on the port 80 from anywhere

Add a rule to allow load balancer redirect request to our container (type = Custom TCP, port = 8080, source anywhere)

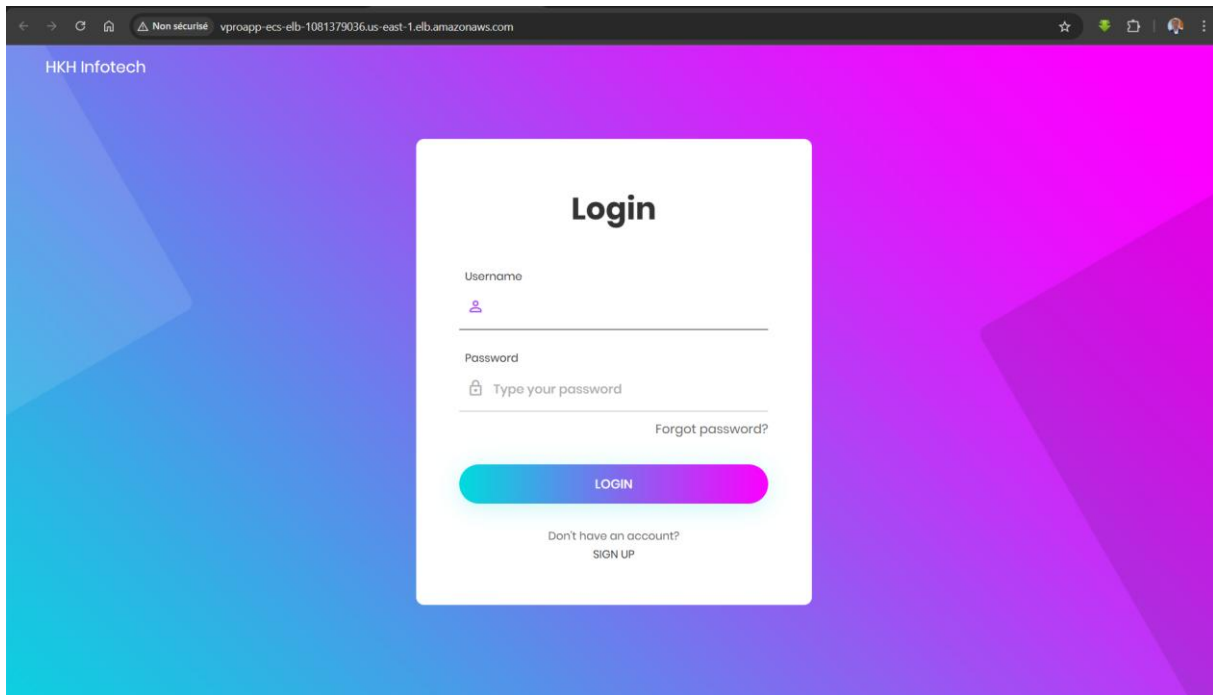
In the load balancing section, check Use Load Balancing

Select Application Load Balancer

In the target group, check create new target group

Give the name vpro-ecs-tg

Click on create



Finally, we have our application running on a container managed in ECS with Fargate service

In these steps, we created the infrastructure and made the deployment manuellement for the first time. In the next section, we will deploy automatically via Jenkins.

Deployment to ECS via Jenkins

To deploy automatically our application via Jenkins, we will add a last stage to our last pipeline. So in the environment variable, we need to conf our service name and the cluster name

```
environment {  
    registryCredential = 'ecr:us-east-1:awscreds'  
    appRegistry = "481665125320.dkr.ecr.us-east-1.amazonaws.com/vprofileappimg"  
    vprofileRegistry = "https://481665125320.dkr.ecr.us-east-1.amazonaws.com"  
    cluster = "vprofile-bis"  
    service = "vprofileappsvc"  
}
```

In next, we add this stage to our pipeline, then build it

```
stage('Deploy to ecs') {  
    steps {  
        withAWS('credentials': 'awscreds', region: 'us-east-1') {  
            sh 'aws ecs update-service --cluster ${cluster} --service ${service} --force-new-deployment'  
        }  
    }  
}
```

To run this last stage, we need to add a last plugin, Pipeline AWS Step

